

Lab 5: Transactions, Contention & Retry Loops (65 min)

Pairs with the [Schema Patterns Playbook](#). Part E is Playbook #5 (Sharded Counter).

Learning Objectives

By the end of this lab you will be able to:

- Trigger SQLSTATE 40001 (`serialization_failure`) from two concurrent sessions
- Build and verify an application-side retry loop with exponential backoff
- Use `AS OF SYSTEM TIME` (follower reads) to bypass contention for reports
- Diagnose contention via the DB Console **Transactions / Insights** pages and via `crdb_internal`
- Apply two patterns that *prevent* contention: sharded counters and `SELECT ... FOR UPDATE` ordering
- Predict whether a given workload will succeed under serializable isolation

Prerequisites

- **Docker Desktop** (or Docker Engine) running — there is no `cockroach` binary to install
- Three terminals open side-by-side (two SQL sessions + one runner)
- Optional but recommended: `python3` with `psycopg2-binary` for the retry-loop script

Setup

```
scripts/crdb up          # start the 3-node cluster (skip if it is already running)
scripts/crdb sql         # open a SQL shell
```

Everything runs in Docker — see [Lab 1](#) for the cluster layout. Use `scripts/crdb.sh` on macOS, Linux, and **WSL** — WSL is Linux, so the shell script is the right one there. Only a native Windows shell (`cmd` or PowerShell) needs `scripts\crdb.bat` .

Capture the connection URL from the demo banner:

```
export CRDB_URL='postgres://root@localhost:26257/?sslmode=disable'
```

Open a second terminal and connect:

```
scripts/crdb sql
```

Now create the lab schema (from either terminal — they share the cluster):

```
CREATE DATABASE bank;
USE bank;

CREATE TABLE accounts (
  id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name    STRING UNIQUE NOT NULL,
  balance DECIMAL(12,2) NOT NULL CHECK (balance >= 0)
);

INSERT INTO accounts (name, balance) VALUES
('Alice',    1000.00),
('Bob',      500.00),
('Charlie',  2000.00);
```

Verify in both terminals:

```
SELECT name, balance FROM accounts ORDER BY name;
```

Tasks

Part A: Produce a 40001 Manually (10 min)

We'll write a contended UPDATE on the same row from two sessions and force a serialization failure.

1. Terminal A — start a transaction and read Alice:

```
BEGIN;
SELECT balance FROM accounts WHERE name = 'Alice';
-- Note the value; do NOT commit yet.
```

2. Terminal B — auto-commit update:

```
UPDATE accounts SET balance = balance - 100 WHERE name = 'Alice';
```

Succeeds immediately. Alice now has 900.

3. Terminal A — try to write based on what you read:

```
UPDATE accounts SET balance = balance - 50 WHERE name = 'Alice';
COMMIT;
```

Expected:

```
ERROR: restart transaction: TransactionRetryWithProtoRefreshError: ...
SQLSTATE: 40001
```

Terminal A's transaction *cannot commit serializably* because Alice changed under it. CockroachDB refuses to lose the write skew.

4. Terminal A — clean up:

```
ROLLBACK;
SELECT name, balance FROM accounts ORDER BY name;
```

Alice = 900, Bob = 500, Charlie = 2000.

Part B: A Working Retry Loop (15 min)

1. Reset balances (any terminal):

```
UPDATE accounts SET balance = CASE name WHEN 'Alice' THEN 1000 WHEN 'Bob' THEN 500 ELSE
2000 END;
```

2. Save `transfer.py` in your working dir:

```
#!/usr/bin/env python3
import os, sys, time, random, psycpg2
from psycpg2 import errors

url = os.environ["CRDB_URL"]
MAX = 5

def transfer(conn, src, dst, amount):
    for attempt in range(1, MAX + 1):
        try:
            with conn:
                # BEGIN ... COMMIT
                with conn.cursor() as cur:
                    cur.execute("UPDATE accounts SET balance = balance - %s WHERE name =
%s",
                                (amount, src))
                    cur.execute("UPDATE accounts SET balance = balance + %s WHERE name =
%s",
                                (amount, dst))
            return attempt
        except errors.SerializationFailure:
            # 40001: exponential backoff with jitter
            time.sleep((2 ** (attempt - 1)) * 0.005 + random.random() * 0.005)
            raise RuntimeError(f"giving up after {MAX} attempts")

conn = psycpg2.connect(url)
attempts = transfer(conn, sys.argv[1], sys.argv[2], int(sys.argv[3]))
print(f"transferred ${sys.argv[3]} {sys.argv[1]} -> {sys.argv[2]} in {attempts}
attempt(s)")
conn.close()
```

3. Bash fallback if you don't have `psycpg2` — `transfer.sh`:

```
#!/usr/bin/env bash
set -euo pipefail
MAX=5
FROM="$1" TO="$2" AMT="$3"
for n in $(seq 1 $MAX); do
    if scripts/crdb sql --execute "
        BEGIN;
        UPDATE accounts SET balance = balance - $AMT WHERE name = '$FROM';
        UPDATE accounts SET balance = balance + $AMT WHERE name = '$TO';
        COMMIT;
    " >/dev/null 2>&1; then
        echo "transferred \$$AMT $FROM -> $TO in $n attempt(s)"
        exit 0
    fi
    sleep 0.0$n
done
echo "giving up after $MAX attempts" >&2
exit 1
```

4. Run them concurrently to force contention:

```
# Terminal A
for i in {1..20}; do python3 transfer.py Alice Bob 10; done
# Terminal B
for i in {1..20}; do python3 transfer.py Bob Alice 10; done
```

Most transfers succeed on attempt 1; some report 2 or 3. None should fail completely.

5. Verify invariants held:

```
SELECT name, balance FROM accounts ORDER BY name;
SELECT sum(balance) FROM accounts; -- should equal starting total
```

Atomicity preserved — money was neither created nor destroyed.

Part C: AS OF SYSTEM TIME — Bypass Contention for Reports (10 min)

Reports that aggregate a big table will often contend with write traffic. The fix: read from a follower at a slightly-stale timestamp.

1. Start a hot write loop in one terminal:

```
while true; do
    scripts/crdb sql \
        --execute "UPDATE accounts SET balance = balance + 1 WHERE name = 'Alice';"
done
```

2. In another terminal, run a current-time aggregate. Time it:

```
\set show_times
SELECT count(*), sum(balance) FROM accounts;
```

Often runs fast; under heavier contention it can stall briefly.

3. Now read at the follower-readable past timestamp:

```
SELECT count(*), sum(balance)
FROM accounts AS OF SYSTEM TIME follower_read_timestamp();
```

This *cannot* contend with the current write loop — it reads at a timestamp from ~4.8 s ago.

4. For an entire transaction:

```
BEGIN;
SET TRANSACTION AS OF SYSTEM TIME follower_read_timestamp();
SELECT name, balance FROM accounts;
SELECT count(*) FROM accounts WHERE balance > 100;
COMMIT;
```

Every statement uses the same historical snapshot. Perfect for reports.

5. Stop the write loop (Ctrl+C).

Part D: SELECT ... FOR UPDATE — Pessimistic Locking (10 min)

For workloads where 40001 retries get expensive, you can take a row lock explicitly. CockroachDB then queues writes instead of retrying.

1. Terminal A:

```
BEGIN;
SELECT balance FROM accounts WHERE name = 'Alice' FOR UPDATE;
```

The `FOR UPDATE` clause acquires a lock on Alice's row.

2. Terminal B — try the same:

```
BEGIN;
SELECT balance FROM accounts WHERE name = 'Alice' FOR UPDATE;
```

This BLOCKS (waits) instead of immediately erroring. Alice's row is locked by A.

3. Terminal A — commit:

```
UPDATE accounts SET balance = balance - 50 WHERE name = 'Alice';
COMMIT;
```

4. Terminal B — observe. Your blocked `SELECT FOR UPDATE` returns the *new* balance (450), and you can now proceed:

```
UPDATE accounts SET balance = balance + 50 WHERE name = 'Alice';
COMMIT;
```

Pessimistic locking trades latency (some sessions wait) for fewer retries. It's the right choice when contention is heavy AND when the work between BEGIN and COMMIT is short.

Part E: Sharded Counter (Playbook #5) — Eliminate the Hotspot Entirely (10 min)

Single-row counters (`UPDATE counters SET n = n + 1`) are a classic CockroachDB anti-pattern. The fix: shard the counter.

1. Create a sharded counter table:

```
CREATE TABLE counter_shards (
  name STRING NOT NULL,
  shard INT NOT NULL,
  n INT NOT NULL DEFAULT 0,
  PRIMARY KEY (name, shard)
);

-- Pre-create shards (avoids contention even on the very first increment)
INSERT INTO counter_shards (name, shard)
SELECT 'page_views', g FROM generate_series(0, 15) g;
```

2. Increment a shard — and note where the shard is chosen:

```
-- The application picks the shard; here it arrives as a literal
UPDATE counter_shards
SET n = n + 1
WHERE name = 'page_views' AND shard = 7;      -- app: randint(0, 15)
```

⚠ **Do not write** `WHERE shard = (random()*16)::INT`. `random()` is a *volatile* function, and a volatile function in a predicate is evaluated **once per row scanned** — so the statement matches a random *number* of shard rows: sometimes zero (the increment is silently lost), sometimes several (it double-counts). Prove it to yourself:

```
SELECT count(*) FROM counter_shards WHERE shard = (random()*16)::INT;  -- run it 8
times
```

Pick the shard in the application, where the value is stable. Lab 8 Part E measures the damage the wrong form does.

3. Read the total — sum across shards:

```
SELECT sum(n) AS total FROM counter_shards WHERE name = 'page_views';
```

4. Drive concurrent increments and watch contention disappear:

```
# 8 concurrent writers, 25 increments each. Note the shape: statements are
# PIPED into 8 long-lived shells rather than spawning 200 processes –
# each `cockroach sql` is a full binary, and 200 of them will exhaust the
# memory on a laptop long before they stress the database.
for w in $(seq 1 8); do
  ( for i in $(seq 1 25); do
      echo "UPDATE counter_shards SET n = n + 1
          WHERE name = 'page_views' AND shard = $((RANDOM % 16));"
    done | scripts/crdb sql >/dev/null 2>&1 ) &
done
wait
```

```
SELECT sum(n) AS total FROM counter_shards WHERE name = 'page_views'; -- must be 200 +
earlier
```

Every increment landed, and contention is spread across 16 rows instead of piling onto one. Compare to incrementing a single-row counter — orders of magnitude better under concurrency.

Part F: Diagnose Contention in DB Console & SQL (10 min)

1. Reproduce contention (terminal A + B concurrently):

```
for i in {1..100}; do python3 transfer.py Alice Bob 5; done &
for i in {1..100}; do python3 transfer.py Bob Alice 5; done &
wait
```

2. **DB Console** → **SQL Activity** → **Transactions**. Look for the contention time column. Click into a high-contention transaction — it shows the blocking transaction and contended row keys.

3. **DB Console** → **SQL Activity** → **Insights**. Failed and slow transactions appear here with one-click drill-in.

4. In SQL:

```
SELECT collection_ts, contention_duration, blocking_txn_id, waiting_txn_id,
       contending_pretty_key
FROM crdb_internal.transaction_contention_events
ORDER BY collection_ts DESC
LIMIT 10;
```

5. Aggregate contention by table:

```
SELECT
  substring(contending_pretty_key from '/Table/(\d+)') AS table_id,
  count(*) AS events,
  sum(contention_duration) AS total_contention
FROM crdb_internal.transaction_contention_events
WHERE collection_ts > now() - INTERVAL '5 minutes'
GROUP BY 1
ORDER BY 3 DESC;
```

Optional — If Time Allows

These parts are not required to complete the lab; they extend it by about 10 minutes. Do them if you finish early, or after class — the cluster and data from the core parts carry over.

Part G: Predict the Behavior (10 min)

For each scenario, predict whether you'll get 40001s, deadlocks, or smooth sailing. Then test.

1. Two sessions, opposite-direction transfers:

```
-- A: Alice -> Bob 100
-- B: Bob -> Alice 100
```

2. Two sessions, ordered-acquisition by name:

```
-- A:
BEGIN;
SELECT balance FROM accounts WHERE name = 'Alice' FOR UPDATE;
SELECT balance FROM accounts WHERE name = 'Bob'   FOR UPDATE;
-- (update + commit)
-- B: same order
BEGIN;
SELECT balance FROM accounts WHERE name = 'Alice' FOR UPDATE;
SELECT balance FROM accounts WHERE name = 'Bob'   FOR UPDATE;
-- (update + commit)
```

Why is this safer than unordered locks?

3. Two sessions, REVERSE-ordered locks:

```
-- A: locks Alice then Bob.
-- B: locks Bob then Alice.
```

CockroachDB *detects* the deadlock and aborts one transaction with an error. Verify.

Cleanup

```
DROP DATABASE bank CASCADE;
```

`\q` exits each SQL shell; the cluster keeps running.

The cluster keeps running between labs — that is the point of it being persistent. To wipe everything and start fresh at any time:

```
scripts/crdb reset
```

Lab 5 Deliverables

□ **40001 triggered manually**: two-session contention demo produced a serializable conflict □ **Retry loop works**: a script with exponential backoff handles 40001 transparently □ **AS OF SYSTEM TIME used**: read-only query runs without contending with concurrent writes □ **SELECT FOR UPDATE applied**: pessimistic locking demonstrated, lock-wait observed □ **Sharded counter built**: removed a single-row write hotspot □ **Contention diagnosed**: via DB Console and via `crdb_internal.transaction_contention_events` □ **Lock-order patterns**: predicted and verified deadlock vs safe behavior

Challenge Exercises

1. **Tune the backoff.** With 4 concurrent transfer processes, measure throughput vs the choice of backoff: constant 1ms, constant 50ms, exponential 1 → 100ms, exponential with jitter. Which wins, and when?
2. **Replace a single-row counter without downtime.** Suppose you have an existing `UPDATE views SET n = n + 1` deployed. Design a migration that swaps to a sharded scheme without changing the app's interface. (*Hint*: views.)
3. **Find the contention pattern.** Trigger the scenarios in Part G and query `transaction_contention_events`. Can you tell which scenario produced the events using only the SQL output (no a-priori knowledge)?

Reference

Symptom	Likely cause	First fix
40001 retry storm	Concurrent writes to same row	App-side retry; consider FOR UPDATE
Long-running read blocks writes	Reader holding leaseholder time	Use AS OF SYSTEM TIME for reports
One node has all the contention	Hot row (counter, queue)	Shard the row
Mysterious "transaction already aborted"	Statement after a failed one in same txn	ROLLBACK and re-run
Deadlock detected	Cross-key lock acquisition order	Always acquire locks in the same order